Celery Notes: Background Tasks Made Simple

Python distributed task queue. Examples use Celery 5.x with Redis as the broker.

Contents

- 1. Why Celery Exists
- 2. The Restaurant Kitchen Analogy
- 3. Core Components
- 4. Quick Start
- 5. Calling Tasks and Reading Results
- 6. Retries, Timeouts and Reliability
- 7. Building Workflows
- 8. Periodic Tasks with Beat
- 9. Inside a Worker
- 10. Concurrency for CPU-bound and I/O-bound Work
- 11. Queues and Routing
- 12. Scaling Up and Out
- 13. Memory Pitfalls
- 14. Monitoring and Debugging
- 15. Production Checklist
- 16. Cheat Sheet

Diagrams appear in sections 1, 3, 7, 9 and 11.

---

1. Why Celery Exists

Suppose a user clicks **Generate Report** and your web app does all the work inside the request. The page hangs, the user stares at a spinner, and that server thread is stuck until the work finishes. Do that for enough users and the whole site stalls.

The fix: do not make the user wait. The web app drops a small message saying 'please do this job' and answers immediately. Something else does the heavy lifting in the background.

Celery is that something else: an asynchronous, distributed **task queue**. It runs units of work outside the request/response cycle, on one machine or on hundreds.

Typical Celery jobs	Why they should not block a request
Sending emails and SMS	Slow third-party APIs
Generating PDFs and reports	CPU heavy, takes seconds or minutes
Image and video processing	CPU heavy
Calling slow external APIs	Network waits
Importing and exporting data	Long running
Scheduled clean-ups and syncs	Nobody is waiting; they run on a clock

---

2. The Restaurant Kitchen Analogy

The whole system maps neatly onto a restaurant.

In the restaurant	In Celery	Job
Waiter	**Web application (producer)**	Takes the order and returns to the customer right away
Order ticket	**Task message**	Describes what to cook (function name and arguments)
Ticket rail	**Broker**	Holds tickets in order until a chef is free
Chefs	**Workers**	Pick up tickets and cook
Head chef	**Main worker process**	Hands tickets to the cooks; does not cook
Pickup counter	**Result backend**	Where finished dishes (return values) wait to be collected
Kitchen clock	**Celery Beat**	Triggers recurring jobs, such as the daily soup prep

Keep this picture in mind. Every later section is a variation on it: more chefs, separate rails for rush orders, and so on.

---

3. Core Components

Celery has four building blocks.

Component	Role	Common choices
Task	A normal Python function marked to run in the background	Any function decorated with `@app.task`
Broker	A queue that holds task messages until a worker takes them	Redis, RabbitMQ
Worker	A background process that pulls tasks from the broker and runs them	`celery worker`
Result backend	*Optional.* Stores task state and return values so you can look them up later	Redis, PostgreSQL, MySQL

Broker choice

	Redis	RabbitMQ
Setup	Very easy, probably already in your stack	Slightly more setup
Delivery guarantees	Good for most apps	Stronger; built for messaging
Best for	Starting out, moderate load	Large or critical pipelines

Do you need a result backend? Only if you read results or task state later. For fire-and-forget jobs such as sending an email, skip it and save the storage.

---

4. Quick Start

```
pip install celery redis
```

```
# tasks.py
from celery import Celery

app = Celery(
    'myapp',
    broker='redis://localhost:6379/0',
    backend='redis://localhost:6379/1',   # optional
)

@app.task
def add(x, y):
    return x + y
```

Start a worker in one terminal:

```
celery -A tasks worker --loglevel=info
```

Call the task from anywhere (another terminal, your web app):

```
from tasks import add

result = add.delay(2, 3)      # returns instantly with an AsyncResult
print(result.id)              # the task ID
print(result.get(timeout=10)) # 5
```

The key idea: `add(2, 3)` runs the function right here, right now. `add.delay(2, 3)` only sends a message; the worker runs it later, probably in a different process or on a different machine.

---

5. Calling Tasks and Reading Results

5.1 Ways to call a task

Call	Meaning
`task(args)`	Normal synchronous call. No Celery involved.
`task.delay(args)`	Shortcut: send to the queue with default options
`task.apply_async(args=[...], ...)`	Full control: delay, queue, expiry, priority

```
from datetime import datetime, timedelta, timezone

send_email.apply_async(args=[user_id], countdown=60)            # run in 60
seconds
send_email.apply_async(args=[user_id], queue='high')            # pick a
queue
send_email.apply_async(args=[user_id], expires=300)             # drop it if
not started in 5 min
send_email.apply_async(args=[user_id],
    eta=datetime.now(timezone.utc) + timedelta(hours=1))        # run at an
exact time
```

5.2 Task lifecycle

A task moves through these states. The result backend records them.

State	Meaning
`PENDING`	Waiting in the queue (also what you see for an unknown task ID)
`STARTED`	A worker picked it up. Only reported if `task_track_started=True`.
`RETRY`	Failed and scheduled for another attempt
`SUCCESS`	Finished; the return value is stored
`FAILURE`	Raised an exception and gave up
`REVOKED`	Cancelled before or during execution

```
res = add.delay(2, 3)
res.state        # 'PENDING', 'SUCCESS', ...
res.ready()      # True when finished
res.get(timeout=10)
```

> **Rule:** never call `.get()` inside another task. It makes a worker wait for another worker and can deadlock. Use workflows (section 7) instead.

---

6. Retries, Timeouts and Reliability

Background jobs fail: networks drop, APIs rate-limit. Plan for it.

Automatic retries with back-off

```
@app.task(
    bind=True,
    autoretry_for=(ConnectionError, TimeoutError),
    retry_backoff=True,                # waits 1s, 2s, 4s, 8s ...
    retry_backoff_max=600,
    retry_jitter=True,                 # randomizes waits so retries do not
stampede
    retry_kwargs={'max_retries': 5},
)
def call_partner_api(self, order_id):
    ...
```

Manual retry

```
@app.task(bind=True, max_retries=3)
def charge(self, order_id):
    try:
        ...
    except TemporaryError as exc:
        raise self.retry(exc=exc, countdown=30)
```

Time limits

```
@app.task(soft_time_limit=50, time_limit=60)
def render_report(report_id):
    ...   # soft limit raises SoftTimeLimitExceeded so you can clean up; hard
limit kills the process
```

Reliability rules

Rule	Why
Make tasks **idempotent** (safe to run twice)	A task can be delivered more than once, for example after a worker crash
Pass **IDs, not objects**	Arguments are serialized to JSON; the database row may also change before the task runs
Use `acks_late=True` for important tasks	The message is removed from the queue only after the task finishes, so a crash means a redelivery instead of a lost job
Set time limits	A stuck task otherwise holds a worker forever
Keep tasks small and focused	Easier to retry, route and monitor

---

7. Building Workflows

Tasks can be combined. First, wrap a call in a **signature** (`.s()`), which is a task call that has not run yet.

Primitive	What it does	Example
chain	Runs tasks one after another; each result feeds the next	`chain(fetch.s(url), parse.s(), save.s())`
group	Runs tasks in parallel	`group(resize.s(img) for img in images)`
chord	A group, then one callback after **all** finish	`chord(group(...))(summary.s())`

```
from celery import chain, group, chord

# fetch -> parse -> save, in order
chain(fetch.s(url), parse.s(), save.s()).apply_async()

# resize 10 images in parallel, then email the user once all are done
chord(group(resize.s(i) for i in image_ids))
(notify_user.s(user_id)).apply_async()
```

`.si()` makes an **immutable** signature that ignores the previous task's result. Use it when the next step should not receive the earlier output.

---

8. Periodic Tasks with Beat

Celery Beat is a small scheduler process. It only *sends* tasks to the queue on a schedule; workers still run them.

```
from celery.schedules import crontab

app.conf.beat_schedule = {
    'clean-expired-sessions': {
        'task': 'tasks.cleanup_sessions',
        'schedule': crontab(hour=2, minute=0),      # every day at 02:00
    },
    'sync-prices': {
        'task': 'tasks.sync_prices',
        'schedule': 30.0,                           # every 30 seconds
    },
}
```

```
celery -A tasks beat --loglevel=info      # run the scheduler
celery -A tasks worker -B                 # dev shortcut: worker + beat
together
```

> **Run exactly one Beat process.** Two copies means every scheduled task is sent twice.

---

9. Inside a Worker

When you start a worker, the process you launched is a **supervisor**. It does not run your tasks itself. It pulls messages from the broker and hands them to a pool of **child processes**.

The plus-one rule

```
Total processes = 1 manager + concurrency (child workers)
```

So `--concurrency=4` gives **5 processes**: 1 manager and 4 children that actually run tasks. When you count RAM or look at `ps`, remember the extra one.

Execution pools

Pool	How it works	Best for
`prefork` (default)	Separate OS processes	CPU-bound work; true parallelism

Pool	How it works	Best for
`threads`	Threads in one process	Light I/O, simple cases
`gevent` / `eventlet`	Thousands of lightweight green threads	Heavy I/O: many API calls, emails, scraping
`solo`	Runs tasks one at a time in the main process	Debugging, tiny setups

```
celery -A tasks worker --concurrency=4                    # prefork, 4
children
celery -A tasks worker -P gevent --concurrency=500        # gevent, I/O
heavy
```

Prefetch: by default each child reserves several messages ahead of time (`worker_prefetch_multiplier`, default 4). With long tasks this lets one worker hoard jobs while others sit idle. Set it to `1` for long-running tasks.

---

10. Concurrency for CPU-bound and I/O-bound Work

More concurrency is not always faster. First decide what your task spends its time doing.

	CPU-bound	I/O-bound
Work	Number crunching, image and video conversion, data processing	Waiting on APIs, databases, email servers, disks
CPU while running	Close to 100% busy	Mostly idle, waiting
Context switching	Costly: interrupts real computation	Nearly free: happens during waits anyway
Best concurrency	About the **number of CPU cores**	**Much higher** than the core count
Best pool	`prefork`	`gevent` or `eventlet`
Kitchen picture	Chefs all chopping at full speed; more chefs than cutting boards just means collisions	Chefs mostly watching pots boil; one chef can keep many pots going

Why too many processes hurts: run 16 busy processes on 8 cores and the operating system keeps swapping them in and out (**context switching**). Time goes into saving and restoring state instead of running your code.

---

11. Queues and Routing

By default every task goes into one queue called `celery`. Real systems separate traffic so a flood of bulk jobs cannot delay urgent ones.

```
app.conf.task_routes = {
    'tasks.send_password_reset': {'queue': 'high'},
    'tasks.send_newsletter':     {'queue': 'bulk'},
    'tasks.render_video':        {'queue': 'heavy'},
}
```

Then start different workers for different queues, each tuned for its job:

```
celery -A tasks worker -Q high  --concurrency=4                 # fast,
small, always free
celery -A tasks worker -Q bulk  -P gevent --concurrency=300     # I/O heavy,
cheap machine
celery -A tasks worker -Q heavy --concurrency=8                 # CPU heavy,
big machine
```

Queue	Traffic	Worker profile
`high`	Urgent, user-facing	Low concurrency, fast nodes, guaranteed to have free capacity
`bulk`	Large batches (newsletters)	High concurrency, budget instances
`heavy`	CPU-heavy jobs	Core-matched concurrency, big machines

---

12. Scaling Up and Out

Strategy	How	Pros	Limits
Vertical	Raise `--concurrency` on one machine	Simple	Capped by that machine's CPU and RAM

Strategy	How	Pros	Limits
Horizontal	Run workers on many machines, all pointing at the same broker	Fault tolerant (others absorb the work if one dies), practically unlimited	More machines to operate

Because workers pull from a shared broker, scaling out is just **starting more workers**. No code changes are needed.

---

13. Memory Pitfalls

The default `prefork` pool gives **every child process its own copy** of your application in RAM.

```
Memory needed = total processes x base application memory
```

Example: a 100 MB app with `--concurrency=8` has 9 processes, so about **900 MB** for one worker.

How to cut it down

Technique	Details
Lazy imports	Import heavy libraries (`pandas` , `torch` , `PIL`) inside the task function, not at the top of the module, so only tasks that need them pay for them
Lean environments	Split worker types into separate services that install only the libraries their tasks need
Switch off unused features	`--without-gossip --without-mingle --without-heartbeat` if you do not use worker-to-worker clustering features
Recycle children	`--max-tasks-per-child=100` or `--max-memory-per-child=200000` (KiB) restarts leaky children

---

14. Monitoring and Debugging

Tools

```
pip install flower
celery -A tasks flower                  # web dashboard on port 5555

celery -A tasks status                  # which workers are alive
celery -A tasks inspect active          # tasks running right now
celery -A tasks inspect reserved        # tasks prefetched but not started
celery -A tasks purge                   # delete all waiting messages
(careful!)
```

Common problems

Symptom	Likely cause	Fix
Task is sent but nothing happens	No worker running, or worker listening to a different queue	Check `celery status` and the `-Q` flag
`NotRegistered` error	Worker never imported the module that defines the task	Add `include=['tasks']` to `Celery(...)` or fix imports
`result.get()` hangs or the state stays `PENDING`	No result backend configured, or the task never ran	Configure `backend=` and check the worker log
`EncodeError` / serialization error	Passing objects that are not JSON-serializable	Pass IDs and plain data
Same task runs twice	Redelivery after a crash, or the Redis `visibility_timeout` (default 1 hour) is shorter than your task or ETA	Make the task idempotent; raise `visibility_timeout` for long ETAs
Worker RAM keeps growing	Memory leak or too many processes	Recycle children, apply section 13
Workers idle while one hoards tasks	Prefetching with long tasks	`worker_prefetch_multiplier = 1`

---

15. Production Checklist

Work through these in order when designing a real pipeline.

1. **Profile the workload.** Is each task CPU-bound or I/O-bound?

2. **Define queues.** Separate urgent work from bulk work.
3. **Choose the worker model.** `prefork` for CPU-bound; `gevent` or `eventlet` for I/O-bound.
4. **Tune concurrency.** Match core count for CPU work; go high for I/O work.
5. **Scale horizontally.** Spread workers across machines for capacity and fault tolerance.

Then confirm the safety basics

- Tasks are idempotent and take IDs, not objects.
- Retries use back-off and jitter, with a maximum.
- Time limits are set on every task.
- `acks_late` is on for tasks you cannot afford to lose.
- Exactly one Beat process runs.
- Secrets and URLs come from environment variables.
- Flower or another monitor is watching queue length and failures.

---

16. Cheat Sheet

```
# define
@app.task(bind=True, acks_late=True, autoretry_for=(Exception,),
retry_backoff=True)
def job(self, item_id): ...

# call
job.delay(1)
job.apply_async(args=[1], queue='high', countdown=30)

# workflow
chain(a.s(1), b.s(), c.s())()
chord(group(a.s(i) for i in ids))(done.s())
```

```
celery -A tasks worker -l info -Q high,bulk -c 4      # start a worker
celery -A tasks worker -P gevent -c 500               # I/O heavy pool
celery -A tasks beat -l info                          # scheduler
celery -A tasks flower                                # dashboard
celery -A tasks inspect active                        # what is running
```

One-line mental model: *the web app drops tickets on a rail (broker), chefs (workers) cook them in the background, and finished dishes wait at the pickup counter (result backend).*

Reference: [Python Celery Explained (YouTube)]